12. Python: Sets

"Removing duplicates, one unique item at a time."

Previous Section:

</> 11. Python: Tuples

Contents:

Another important data structure in Python is the set.

At first glance, sets may look similar to lists or tuples because they can store multiple values. However, sets are designed for a very different purpose.

Instead of focusing on order, sets focus on uniqueness, fast membership checking, and mathematical set operations.

This makes sets extremely useful when we want to remove duplicates, compare collections, or perform operations such as union and intersection.

1. What is a Set?

A set stores an unordered collection of unique elements.

Key properties of sets:

- Sets have no order, meaning we cannot retrieve elements using indexes.

- Sets automatically remove duplicate values.
- Sets are enclosed using curly brackets `{}`.

Example:

```
set1 = {1, 2, 3, 4, 5}
print(set1)
```

{1, 2, 3, 4, 5}

Notice how duplicates are removed automatically:

```
numbers = {1, 2, 2, 3, 3, 4, 5}
print(numbers)
```

{1, 2, 3, 4, 5}

Since sets are unordered, indexing does not work:

```
set1 = {10, 20, 30}
print(set1[0])  # Error
```

TypeError: 'set' object is not subscriptable

Instead, we usually loop through the set:

```
set1 = {'Apple', 'Google', 'Microsoft'}

for company in set1:
    print(company)
```

Google
Apple
Microsoft

We can also create sets using the `set()` constructor.

```
set1 = set([1, 2, 3, 4])
print(set1)
```

{1, 2, 3, 4}

However, be careful with empty curly brackets `{}`. In Python:

```
empty_data = {}
print(type(empty_data))
```

<class 'dict'>

This creates a **dictionary**, not a set. To create an empty set, we must use:

```
empty_set = set()
print(type(empty_set))
```

<class 'set'>

We can also convert lists or tuples into sets. One very useful feature is that sets automatically remove duplicate values.

```
# Example with a list:
numbers = [1, 2, 2, 3, 3, 4, 5]
set_numbers = set(numbers)
print(set_numbers)

# Example with a tuple:
tuple_data = (10, 10, 20, 30, 30, 40)
set_data = set(tuple_data)
print(set_data)
```

{1, 2, 3, 4, 5}
{40, 10, 20, 30}

This is one of the most common ways to quickly remove duplicates from a list or tuple in Python.

2. Methods in Sets

Sets provide many useful methods for adding, removing, comparing, and combining data.

- `set.add()` : Adds a single element into the set.

```
set1 = {1, 2, 3}
set1.add(4)

print(set1)
```

{1, 2, 3, 4}

- `set.clear()` : Removes all elements from the set.

```
set1 = {1, 2, 3}
set1.clear()

print(set1)
```

set()

- `set.discard()`: Removes a specific element if it exists. Unlike `remove()`, it does not produce an error if the element does not exist.

```
set1 = {1, 2, 3, 4}
set1.discard(3)

print(set1)
```

{1, 2, 4}

- `set.pop()`: Removes and returns an arbitrary element from the set. Because sets are unordered, we do not know which element will be removed.

```
set1 = {10, 20, 30}
value = set1.pop()

print("Removed:", value)
print(set1)
```

```
Removed: 10
{20, 30}
```

- `set.remove()` : Removes a specific element. Unlike `discard()`, this method produces an error if the element does not exist.

```
set1 = {1, 2, 3, 4}
set1.remove(2)

print(set1)
```

```
{1, 3, 4}
```

- `set.update()` : Adds all elements from another iterable into the set.

```
set1 = {1, 2, 3}
set1.update([4, 5, 6])

print(set1)
```

```
{1, 2, 3, 4, 5, 6}
```

- `set.difference()`: Returns elements that exist in the first set but not in the second set.

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
diff = set1.difference(set2)

print(diff)
```

{1, 2}

- `set.intersection()`: Returns the common elements between sets.

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
inter = set1.intersection(set2)

print(inter)
```

{3, 4}

- `set.union()` : Returns all unique elements from both sets.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
union_set = set1.union(set2)

print(union_set)
```

{1, 2, 3, 4, 5}

- `set.isdisjoint()` : Returns `True` if two sets have no common elements.

```
set1 = {1, 2, 3}
set2 = {10, 20, 30}

print(set1.isdisjoint(set2))
```

True

- `set.issubset()` : Returns `True` if all elements of the current set exist in another set.

```
set1 = {1, 2}
set2 = {1, 2, 3, 4}

print(set1.issubset(set2))
```

True

- `set.issuperset()` : Returns `True` if the current set contains all elements of another set.

```
set1 = {1, 2, 3, 4}
set2 = {1, 2}

print(set1.issuperset(set2))
```

True

Built-in Functions with Sets

We can also use several built-in functions with sets.

- `len(set)` : Returns the number of elements.

```
set1 = {1, 2, 3, 4}
print(len(set1))
```

4

- `sum(set)` : Returns the sum of all elements.

```
set1 = {1, 2, 3, 4}
print(sum(set1))
```

10

- `max(set)` and `min(set)`

Returns the maximum and minimum values.

```
set1 = {5, 10, 15, 20}
print(max(set1))
print(min(set1))
```

20
5

One of the biggest advantages of sets is that membership checking is very fast:

```
numbers = {1, 2, 3, 4, 5}
print(3 in numbers)
print(10 in numbers)
```

True
False

Summary

Sets are unordered collections that store only unique values. They are especially useful when we want to remove duplicates, perform mathematical set operations, compare collections, or quickly check whether an element exists.

Unlike lists and tuples: sets do not support indexing, and elements are not stored in a fixed order.

Overall, sets are one of the most powerful data structures in Python for handling unique data efficiently.

References

https://www.geeksforgeeks.org/python/sets-in-python/

https://www.programiz.com/python-programming/set

https://realpython.com/python-sets/

Next Section:

</> 13. Python: Dictionaries